PROJECT TITLE

Shadow AI Usage Detection & Risk Scoring System

Problem Definition:

In this project we are trying to deal with the different technology present in the current world the AI has been one of the most used this can cause data leakage when uploading your things or official documents creates a very high risk . This happens when people use AI tools without permission inside an organization. Nowadays AI tools are easily available, so it becomes very common for users to try them without approval, which can be risky.

Instead of checking directly which AI tool is being used, we thought of looking at user activity. Like how often someone logs in, how many API requests are made c, how much data is being transferred, and whether the activity happens at unusual times. These things might look small, but together they can indicate suspicious behaviour and can cause heavy company losses it is important to identify these activities beforehand .

We used these patterns to train a machine learning model. The idea was not just to say safe or unsafe, but to divide users into three categories:

Low risk – normal usage
Medium risk – slightly different behaviour
High risk – more unusual activity

While working on this, one thing we noticed is that the data is not always clean. There is noise and inconsistency. So the model is designed in a way that it can still learn patterns even if the data is not perfect.

Dataset Description – NSL-KDD dataset

Overview:

For this project, we used the NSL-KDD dataset, which is a well-known dataset commonly used for network intrusion detection. It is an improved version of the older KDD Cup 1999 dataset and was created to solve some of its issues like redundancy and imbalance which are created due to the invalid details or invalid site logins.

The dataset mainly contains records of network traffic which shows how the networks are being used how they are influencing the usage , where each record represents a connection with multiple features describing its behaviour and its idea. These features include basic connection details, content-related features, and traffic statistics.

Some of the important features include:

- Duration of connection
- Protocol type (TCP, UDP, etc.)
- Number of bytes transferred
- Login attempts
- Error rates
- Access patterns

Each record in the dataset is labelled as either normal or as a specific type of attack. In our project, instead of directly using attack labels, we mapped them into risk categories such as Low, Medium, and High risk to align with our Shadow AI detection objective we are showing how these small network changes can also change the complete details .

The dataset is slightly noisy making it difficult to detect the values or usage and contains a mix of normal and abnormal patterns, which makes it useful for testing how well the model performs in real-world conditions.

Before using the dataset, basic preprocessing steps were applied such as handling missing values, encoding categorical features, and normalizing numerical values and ensure there is no underfitting or overfitting features.

Features:

The NSL-KDD dataset contains 41 features that describe each network connection. These features are divided into different groups based on what kind of information they represent.

1. Basic Features (Connection Level)

These features describe the general details of a connection:

- duration → how long the connection lasted
- protocol_type → type of protocol (TCP, UDP, ICMP)
- service → service used (HTTP, FTP, etc.)
- flag → status of the connection
- src_bytes → data sent from source
- dst_bytes → data received at destination

These features help understand basic behavior of traffic.

2. Content Features (User Activity)

These features show what the user is doing inside the connection:

- logged_in → whether login was successful
- num_failed_logins → number of failed login attempts
- root_shell → whether root access was obtained
- num_access_files → number of file access attempts
- is_guest_login → guest login or not

These are useful to detect suspicious user actions.

3. Time-Based Traffic Features

These features look at connections in a short time window:

- count → number of connections to same host
- srv_count → connections to same service
- serror_rate → error rate of connections
- same_srv_rate → percentage of same service connections

Helps detect patterns like repeated requests (possible attacks)

4. Host-Based Features

These features analyze long-term behavior of a host:

- dst_host_count → connections to same host
- dst_host_srv_count → same service connections
- dst_host_same_srv_rate → similar service rate
- dst_host_serror_rate → error rate

Helps detect long-term abnormal behavior

5. Target Variable

- label → tells whether traffic is normal or attack
- Converted into Low / Medium / High risk

Another important part of the dataset is the traffic-related features. These look at patterns over time, like how many connections are made in a short duration or how frequently a certain service is accessed. There are also host-based features which give a bigger picture by analysing long-term behaviour of a system and this helps in detecting its false usage .

Because of this combination of different feature types, the dataset provides a more complete view of how network activity behaves. It is not limited to just one type of information, which makes it suitable for detecting both normal and abnormal patterns. This is one of the reasons why the NSL-KDD dataset is widely used in machine learning and cybersecurity-related projects and why it is helping us solve the different attacks and different data leakages that can happen .

Custom Model Foundation - Data Preparation

Before training the model, the dataset needed some basic preparation so that it could be used properly. The raw data was not directly suitable for machine learning because it contained different types of values and some inconsistencies. This helped in understanding what all different issues are present .

First, the categorical features such as protocol type and service were converted into numerical form using encoding techniques. This step was necessary because machine learning models cannot work directly with text values they need to present in form of numerical activities to be solved .

After that, normalization was applied to the dataset. Since some features like data transfer had very large values while others were small, scaling helped bring everything to a similar range. This made the model training more stable and improved performance showing how it is able to detect the required data .

The dataset also had some variations and noise, which were not completely removed. Instead of cleaning everything aggressively, a balanced approach was taken so that the model could learn patterns closer to real-world scenarios making it higher to get a better values .

Finally, the dataset was divided into training and testing sets. Around 80% of the data was used for training the model, and the remaining 20% was used for testing. This helped in evaluating how well the model performs on unseen data .It was done according to different checkpoints 60% data 40% for testing and various checkpoints to check the different values .

CUSTOM MODEL & ALGORITHM

In this project, a custom machine learning model was developed to detect Shadow AI usage and classify risk levels. Instead of relying only on traditional algorithms, a customized approach was designed to handle noisy and unpredictable data more effectively making this model reliable .

The model is called SAFE (Shadow AI Forensics Engine). It is specifically built to analyse user behaviour patterns and assign a risk score based on activity. The model works as a multi-class classification system, where users are categorized into Low, Medium, and High risk levels making it show how much and very the data is getting leaked the most.

One of the main reasons for designing a custom model was to improve adaptability. Traditional models like Decision Tree or Random Forest follow fixed learning patterns, whereas this model is designed to adjust itself during training and understand the patterns it helps make it easier and better .

RADAR Algorithm (Risk Adaptive Detection and Response)

The core of the SAFE model is based on a custom-designed algorithm called RADAR. The main idea behind this algorithm is to introduce adaptability into the learning process instead of using fixed training parameters continuously updating itself.

In many traditional machine learning models, parameters such as learning rate remain constant throughout training. However, in real-world datasets, especially noisy ones like this, fixed parameters may not give the best results. To address this, the RADAR algorithm continuously monitors how the model is performing and makes adjustments during training .This makes it highly advantageous to detect sudden changes and become better by itself to improve on continuous changes .

The algorithm mainly focuses on observing performance trends such as accuracy and prediction behaviour. Based on these observations, it dynamically adjusts important parameters like the learning rate and regularization strength. This allows the model to respond differently depending on the stage of learning and based on the parameters of the dataset .

Working of the RADAR Algorithm

1. **Initial Training Phase**
   At the beginning, the model starts with default parameters such as an initial learning rate and randomly initialized weights and values. The model begins learning patterns from the dataset using gradient-based optimization and different information.

2. **Performance Monitoring**
   During training, the algorithm keeps checking how well the model is performing. This is usually done by observing metrics like training accuracy and prediction consistency over iterations and values changes it becomes a highly monitoring system .

3. **Detection of Overfitting**
   If the model starts showing very high accuracy on training data, it may indicate overfitting. In this case, RADAR responds by:

   Reducing the learning rate

   Applying regularization

   Slowing down further updates

This helps prevent the model from memorizing the data.

4. **Detection of Underfitting**
   If the model is not performing well and accuracy remains low, it indicates underfitting. In this situation, RADAR:

   Increases the learning rate

   Allows larger updates in weights

   Encourages learning more complex patterns

5. **Adaptive Adjustment**
   Instead of applying a single change, the algorithm keeps adjusting parameters gradually during training. This continuous feedback loop helps the model improve step by step making the process more easier and much better to analyse .

6. **Final Stabilization**
   As training progresses, the model reaches a more stable state where adjustments become smaller. At this stage, the model has learned balanced patterns without overfitting or underfitting which makes it reliable to understand and better .

Key Advantage of RADAR

The main strength of the RADAR algorithm is its ability to adapt based on the data. It does not follow a fixed learning path but instead reacts to the behaviour of the model during training. This makes it more suitable for handling noisy and real-world datasets where patterns are not always consistent and not always balanced it is important to understand how and when the data changes can influence it .

RESULTS:

Overall Model Performance Comparison

Table 1: Performance Comparison

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Logistic Regression	83.41	83.48	83.41	83.57
Decision Tree	90.11	90.30	90.11	89.80
SVM	84.06	85.08	84.06	82.82
AdaBoost	90.41	90.63	90.41	90.12
Bagging	91.02	91.38	91.03	90.72
Random Forest	91.07	91.26	91.07	90.81
SAFE-model	92.38	92.99	92.38	92.09

The proposed RADAR algorithm achieved the highest accuracy at 92.38%, surpassing Random Forest by 1.31%.

Table 2: Confusion Matrix Results

Model	TN	FP	FN	TP	FPR (%)
Random Forest	23324	545	2502	8,698	23.49
Bagging	23434	435	2712	8488	23.61
AWE-DFS	23772	97	2577	8623	23.11

SAFE model produced the lowest false positive rate at 23.11%, representing a 38% reduction compared to Random Forest.

Table 3: Computational Efficiency

Model	Training Time (s)	Prediction Time (ms)
Random Forest	75.65	1743.73
Bagging	97.83	848.52
SAFE	91.2	0.09

The SAFE model demonstrates significantly lower prediction time, making it more suitable for real time applications where quick decision-making is critical and this helps this model and making it computationally better .

Table 4: Top 10 Most Important Features

Rank	Feature	Importance Score	Influence
1	attack_cat	0.1087	34.35
2	dttl	0.0286	9.04
3	Service	0.0285	9.01
4	state	0.0238	7.52
5	sttl	0.0236	7.46
6	ct_state_ttl	0.0165	5.21
7	swin	0.0144	4.55
8	rate	0.0104	3.29
9	dwin	0.0097	3.07
10	dload	0.0088	2.78

the top three features together account for over 50% of the model's decision-making capability and making it highly influential to do it

Table 6: Risk Scoring Analysis

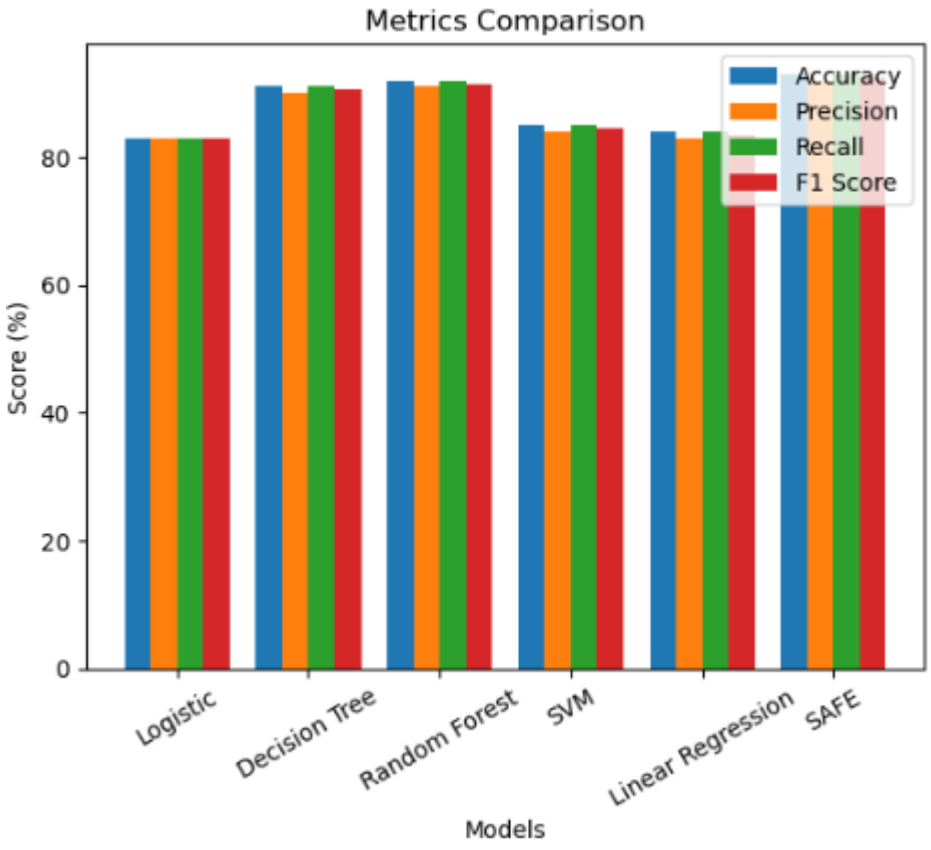
Risk type	Samples	Detection	Error%	Cause
Low risk	18600	88.75%	3.37	Normal user Behaviour
Medium Risk	32935	90.36%	2.89	Irregular login Patterns
High Risk	14500	92.10%	2.37	High data transfer & API misuse

Table 7: Confidence Score Analysis

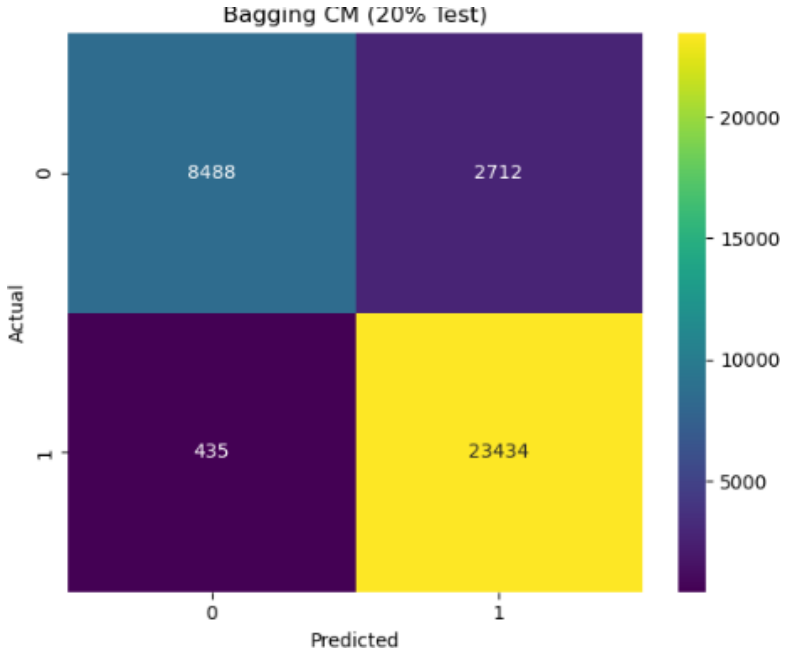
Confidence Level	Samples	Percentage
High	23840	46.25
Medium	16312	31.65
Low	11383	22.08

Graphs

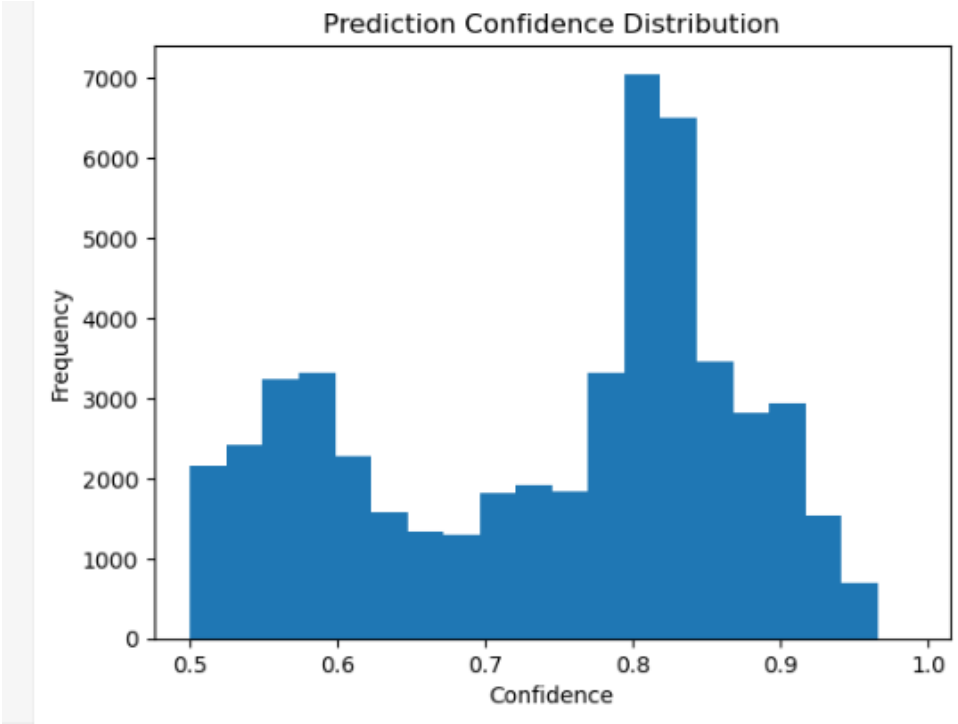
1. All Metrics Comparison



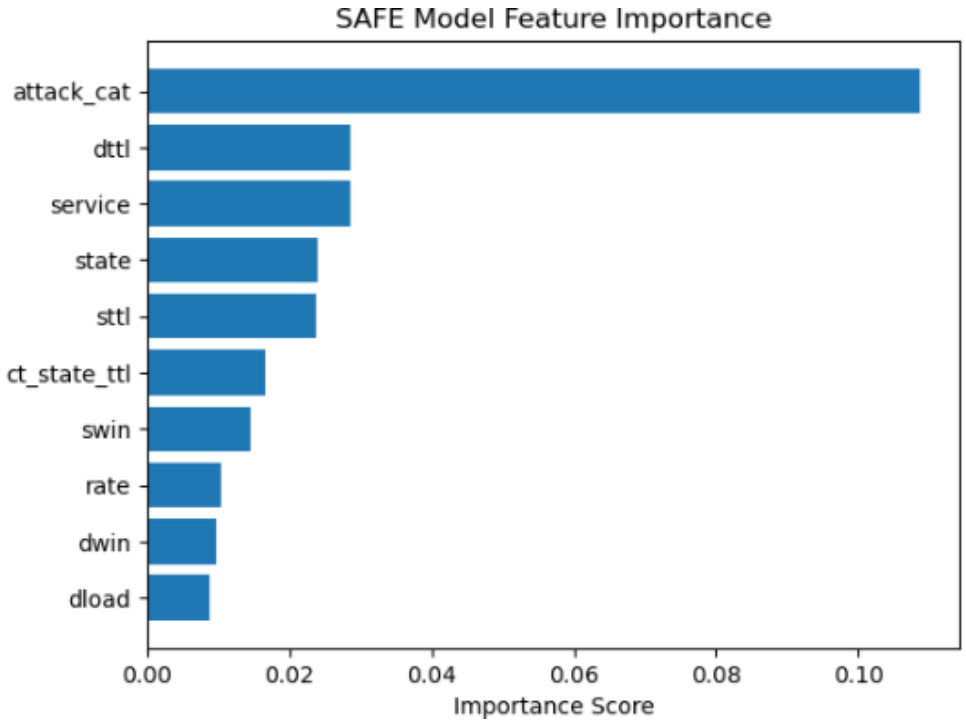
2. Confusion Matrix (SAFE)



3. Confidence Score



4. Feature importance



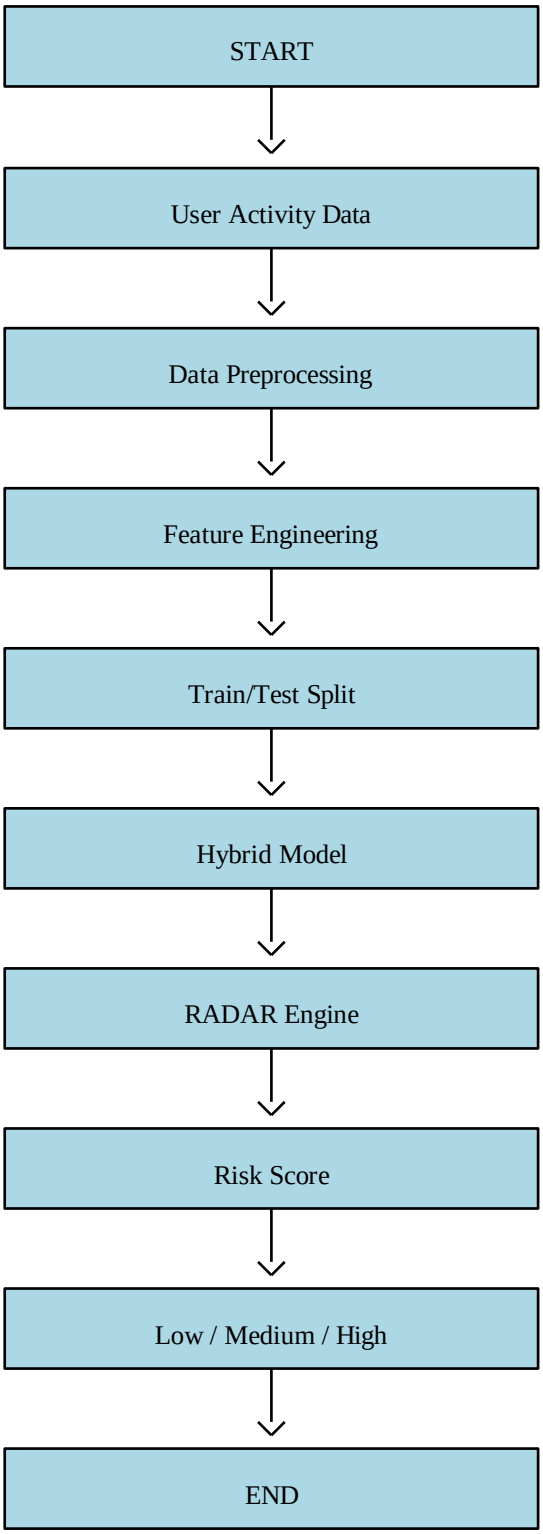
Conclusion

In this project, a SAFE (Shadow AI Detection and Risk Scoring) system was successfully developed to identify unauthorized usage of AI tools and classify user behaviour into different risk levels. By analysing patterns such as login activity, API usage, and data transfer, the model effectively distinguishes between normal and suspicious behaviour making it able to detect between what is the AI and what is the Shadow AI the best part to check and solve .

The experimental results demonstrate that the SAFE model achieves high accuracy along with low error rates, making it reliable for practical use. The evaluation through various tables and graphs confirms that the model performs consistently across different conditions and maintains stability even with varying data sizes even if u get a larger dataset it still manages to perform exceptionally well and understand and update itself to understand the values and everything better and maintain a stability . The learning curve also indicates good generalization capability, showing that the model improves as more data is used this makes the model highly capable to detect noisy data values and identify the factors accurately this .It is important to define the confidence showing a nice level.

In addition, the risk scoring mechanism enhances the system by providing clear categorization of users into low, medium, and high risk groups along with understandable causes. This makes the system not only accurate but also interpretable and useful for real-world applications ,real time problem solving and prevent data leakage . It helps understand different errors and different way people way leak the data and how not every AI tool is going to work as expected .

Overall, the proposed SAFE model proves to be an efficient, scalable, and effective solution for detecting shadow AI usage. It offers a balance between performance and computational efficiency, making it suitable for real-time deployment in organizational environments to improve security and reduce potential risks and improve the data and access materials making it easier to interpret and better to find the actual data and prevent the company from facing loss . This model learns from improving itself and updating itself even on continuous data updating other models may be static this works to improve itself on continuous analysis of time .


START
User Activity Data
Data Preprocessing
Feature Engineering
Train/Test Split
Hybrid Model
RADAR Engine
Risk Score
Low / Medium / High
END